▶ 작업 보고서: 게시글 페이징 단계별 학습 - Step 2

- **작업 일시**: 2026-03-20
- **진행 단계**: 완료 (Step 2: SQL 기초 페이징 LIMIT/OFFSET)

1. 🌊 전체 작업 흐름 (Workflow)

```
1. [Repository] JPQL 기반의 페이징 쿼리 메서드 'findAll(limit, offset)' 추가
2. [Service] '게시글목록보기(page)'로 메서드 파라미터 확장 및 OFFSET 계산 로직 구현
3. [Controller] '/board/list'에서 'page' 쿼리 파라미터 수신 및 현재 페이지 정보 모델에 추가
4. [View] 화면에 현재 페이지 정보를 표시하여 파라미터에 따른 결과 변화 확인
```

UI Mockup

```
+-------------------------------------------------------+
| [Blog] (Navbar)  글쓰기  회원정보수정  로그아웃        |
+-------------------------------------------------------+
|                                                       |
|      +-----------------------------------------+      |
|      |        게시글 목록 (현재 페이지: 1)      |      |
|      |-----------------------------------------|      |
|      |  번호  |  제목                |  상세보기 |      |
|      |-----------------------------------------|      |
|      |  17    |  열일곱 번째 게시글  | [상세보기]|      |
|      |  16    |  열여섯 번째 게시글  | [상세보기]|      |
|      |  15    |  열다섯 번째 게시글  | [상세보기]|      |
|      +-----------------------------------------+      |
|                                                       |
|      (URL: /board/list?page=1 로 접속 시 위와 같이 나옴) |
|                                                       |
+-------------------------------------------------------+
```

2. 🧩 변경된 모든 코드 포함

1) Repository: `BoardRepository.java`

JPQL의 `LIMIT`와 `OFFSET`을 직접 사용하는 쿼리를 정의했습니다.

```java
public interface BoardRepository extends JpaRepository<Board, Integer> {
    // ...
    @Query("SELECT b FROM Board b ORDER BY b.id DESC LIMIT :limit OFFSET :offset")
    List<Board> findAll(@Param("limit") int limit, @Param("offset") int offset);
}
```

2) Service: `BoardService.java`

전체 조회가 아닌, 건너뛸 개수(OFFSET)를 계산하여 필요한 만큼만 가져오도록 변경했습니다.

```
public List<BoardResponse.ListDTO> 게시글목록보기(int page) {
    int limit = 3; // 한 페이지당 출력 개수 (고정)
    int offset = page * limit; // 건너뛸 데이터 수 계산

    // DB에서 해당 범위의 데이터만 가져옴
    var boardList = boardRepository.findAll(limit, offset);
    return boardList.stream()
            .map(BoardResponse.ListDTO::new)
            .collect(Collectors.toList());
}
```

3) Controller: `BoardController.java`

URL로부터 페이지 번호를 받아 서비스에 전달하고, 현재 페이지를 뷰로 넘깁니다.

```
@Controller
public class BoardController {
    // ...
    @GetMapping("/board/list")
    public String list(@RequestParam(defaultValue = "0") int page, Model model) {
        // 현재 페이지에 해당하는 데이터만 가져오기
        var boardList = boardService.게시글목록보기(page);
        model.addAttribute("boardList", boardList);
        model.addAttribute("page", page); // 현재 페이지 정보를 뷰에 전달
        return "board/list";
    }
}
```

4) View: `list.mustache`

현재 사용자가 몇 페이지에 있는지 확인할 수 있는 문구를 추가했습니다.

```
<div class="container p-5">
    <h3>게시글 목록 (현재 페이지: {{page}})</h3>
    <!-- 테이블 생략 (전 단계와 동일) -->
</div>
```

3. 🍪 상세비유 쉬운 예시 (Easy Analogy)

"이번 작업은 **도서관 게시판에 붙일 목록을 '몇 번부터 몇 번까지' 적을지 정한 것**과 같습니다. 게시판 자리가 좁아서 한 번에 3권만 적을 수 있다고 가정해 봅시다.

- 0페이지: 책 1번부터 3개를 적음 (건너뛰기 0)
- 1페이지: 책 1~3번은 이미 적었으니 3개를 건너뛰고, 4번부터 3개를 적음 (건너뛰기 3)
- 2페이지: 책 1~6번은 건너뛰고, 7번부터 3개를 적음 (건너뛰기 6) 이것이 바로 `LIMIT 3 OFFSET (page * 3)`의 원리입니다!"

4. 📚 기술 딥다이브 (Technical Deep-dive)

- **LIMIT**:
  - 조회할 결과의 최대 개수를 제한합니다. 페이징 처리 시 '한 페이지에 보여줄 데이터의 크기' 역할을 합니다.
- **OFFSET**:
  - 결과 집합의 시작 위치를 지정합니다. `OFFSET 5`이면 상위 5개를 건너뛰고 6번째 데이터부터 조회를 시작합니다.
- **JPQL과 페이징**:
  - Hibernate 6부터 JPQL 내에서 `LIMIT`와 `OFFSET` 키워드를 직접 사용할 수 있게 되었습니다. 이전 버전에서는 `Query.setFirstResult()`, `setMaxResults()` 메서드를 사용해야 했습니다.
- **@RequestParam(defaultValue = "0")**:
  - 클라이언트가 `?page=1`과 같이 명시적으로 페이지를 주지 않아도, 서버에서 기본값(0)을 설정하여 에러 없이 처리할 수 있게 돕습니다.